FLUTTER LAB SHEET – 10

Flutter Firebase Authentication and Cloud Firestore

Complete solutions for all 15 programs

Prepared for: Shreya Paliwal

Note: These examples use FlutterFire. Before running them, create a Firebase project, enable Email/Password sign-in, create a Cloud Firestore database, and configure the app using FlutterFire CLI.

Common setup (required)

```
1. Create a Flutter project:
   flutter create firebase_lab
   cd firebase_lab

2. Add packages:
   flutter pub add firebase_core firebase_auth cloud_firestore

3. Configure Firebase (install Firebase CLI and FlutterFire CLI first):
   dart pub global activate flutterfire_cli
   flutterfire configure

4. In Firebase Console:
   - Authentication > Sign-in method > enable Email/Password.
   - Firestore Database > Create database.

5. Use the generated firebase_options.dart file. Do not write your Firebase keys manually.
```

The examples below are individual main.dart implementations. Replace the contents of lib/main.dart with the relevant program. The generated lib/firebase_options.dart must remain in the project.

1. Connect Flutter application to Firebase

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(child: Text('Firebase Connected')),
      ),
    );
  }
}
```

2. Register a new user with email and password

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: RegisterPage()));
}

class RegisterPage extends StatefulWidget {
  const RegisterPage({super.key});
  @override
  State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final email = TextEditingController();
  final password = TextEditingController();

  Future<void> register() async {
    try {
      await FirebaseAuth.instance.createUserWithEmailAndPassword(
        email: email.text.trim(),
        password: password.text.trim(),
      );
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Registration successful')),
      );
```

```
    } on FirebaseAuthException catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.message ?? 'Registration failed')),
      );
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Register')),
    body: Padding(
      padding: const EdgeInsets.all(20),
      child: Column(children: [
        TextField(controller: email,
          decoration: const InputDecoration(labelText: 'Email')),
        TextField(controller: password, obscureText: true,
          decoration: const InputDecoration(labelText: 'Password')),
        ElevatedButton(onPressed: register, child: const Text('Register')),
      ]),
    ),
  );
}
```

3. Log in using signInWithEmailAndPassword()

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: LoginPage()));
}

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});
  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final email = TextEditingController();
  final password = TextEditingController();

  Future<void> login() async {
    try {
      await FirebaseAuth.instance.signInWithEmailAndPassword(
        email: email.text.trim(),
        password: password.text.trim(),
      );
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Login successful')),
      );
    } on FirebaseAuthException catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.message ?? 'Login failed')),
      );
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Login')),
    body: Padding(
      padding: const EdgeInsets.all(20),
      child: Column(children: [
        TextField(controller: email,
          decoration: const InputDecoration(labelText: 'Email')),
        TextField(controller: password, obscureText: true,
          decoration: const InputDecoration(labelText: 'Password')),
        ElevatedButton(onPressed: login, child: const Text('Login')),
      ]),
    ),
  );
}
```

4. Display logged-in user's email and Logout button

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
```

```
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: UserHomePage()));
}

class UserHomePage extends StatelessWidget {
  const UserHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    final user = FirebaseAuth.instance.currentUser;
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Text('Logged in as: ${user?.email ?? "No user"}'),
          ElevatedButton(
            onPressed: () async {
              await FirebaseAuth.instance.signOut();
              if (context.mounted) {
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(content: Text('Logged out')),
                );
              }
            },
            child: const Text('Logout'),
          ),
        ],
      )),
    );
  }
}
```

5. Show authentication errors using SnackBar

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: AuthErrorPage()));
}

class AuthErrorPage extends StatefulWidget {
  const AuthErrorPage({super.key});
  @override
  State<AuthErrorPage> createState() => _AuthErrorPageState();
}

class _AuthErrorPageState extends State<AuthErrorPage> {
  final email = TextEditingController();
  final password = TextEditingController();

  Future<void> register() async {
    try {
      await FirebaseAuth.instance.createUserWithEmailAndPassword(
        email: email.text.trim(),
        password: password.text.trim(),
      );
      showMessage('Account created successfully');
    } on FirebaseAuthException catch (e) {
      String message;
      switch (e.code) {
        case 'email-already-in-use':
          message = 'This email is already registered';
          break;
        case 'weak-password':
          message = 'Password is too weak';
          break;
        case 'invalid-email':
          message = 'Enter a valid email';
          break;
        default:
          message = e.message ?? 'Authentication error';
      }
      showMessage(message);
    }
```

```
  }

  void showMessage(String message) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(message)),
    );
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Authentication Errors')),
    body: Padding(
      padding: const EdgeInsets.all(20),
      child: Column(children: [
        TextField(controller: email,
          decoration: const InputDecoration(labelText: 'Email')),
        TextField(controller: password, obscureText: true,
          decoration: const InputDecoration(labelText: 'Password')),
        ElevatedButton(onPressed: register,
          child: const Text('Register')),
      ]),
    ),
  );
}
```

6. Reset password using sendPasswordResetEmail()

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: ResetPasswordPage()));
}

class ResetPasswordPage extends StatefulWidget {
  const ResetPasswordPage({super.key});
  @override
  State<ResetPasswordPage> createState() => _ResetPasswordPageState();
}

class _ResetPasswordPageState extends State<ResetPasswordPage> {
  final email = TextEditingController();

  Future<void> resetPassword() async {
    try {
      await FirebaseAuth.instance.sendPasswordResetEmail(
        email: email.text.trim(),
      );
      showMessage('Password reset email sent');
    } on FirebaseAuthException catch (e) {
      showMessage(e.message ?? 'Unable to send reset email');
    }
  }

  void showMessage(String message) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(message)),
    );
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Reset Password')),
    body: Padding(
      padding: const EdgeInsets.all(20),
      child: Column(children: [
        TextField(controller: email,
          decoration: const InputDecoration(labelText: 'Email')),
        ElevatedButton(onPressed: resetPassword,
          child: const Text('Send Reset Link')),
      ]),
    ),
  );
}
```

7. Show Login or Home screen using authStateChanges()

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';
```

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: AuthGate()));
}

class AuthGate extends StatelessWidget {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<User?>(
      stream: FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Scaffold(
            body: Center(child: CircularProgressIndicator()),
          );
        }
        if (snapshot.hasData) return const HomeScreen();
        return const LoginScreen();
      },
    );
  }
}

class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Login Screen')),
    body: Center(child: ElevatedButton(
      onPressed: () async {
        // Use a registered account for this demonstration.
        await FirebaseAuth.instance.signInWithEmailAndPassword(
          email: 'student@example.com',
          password: 'password123',
        );
      },
      child: const Text('Login with demo account'),
    )),
  );
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Home Screen')),
    body: Center(child: Text(
      'Welcome ${FirebaseAuth.instance.currentUser?.email}',
    )),
  );
}
```

8. Add a student record to Cloud Firestore

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: AddStudentPage()));
}

class AddStudentPage extends StatefulWidget {
  const AddStudentPage({super.key});
  @override
  State<AddStudentPage> createState() => _AddStudentPageState();
}

class _AddStudentPageState extends State<AddStudentPage> {
  final name = TextEditingController();
  final roll = TextEditingController();
  final course = TextEditingController();

  Future<void> addStudent() async {
    await FirebaseFirestore.instance.collection('students').add({
      'name': name.text.trim(),
      'rollNumber': roll.text.trim(),
      'course': course.text.trim(),
      'createdAt': FieldValue.serverTimestamp(),
```

```
    });
    name.clear(); roll.clear(); course.clear();
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Student added')),
    );
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Add Student')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(children: [
        TextField(controller: name,
          decoration: const InputDecoration(labelText: 'Name')),
        TextField(controller: roll,
          decoration: const InputDecoration(labelText: 'Roll Number')),
        TextField(controller: course,
          decoration: const InputDecoration(labelText: 'Course')),
        ElevatedButton(onPressed: addStudent,
          child: const Text('Save Student')),
      ]),
    ),
  );
}
```

9. Display all students in real time using StreamBuilder and ListView

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: StudentListPage()));
}

class StudentListPage extends StatelessWidget {
  const StudentListPage({super.key});

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('All Students')),
    body: StreamBuilder<QuerySnapshot>(
      stream: FirebaseFirestore.instance
          .collection('students').snapshots(),
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return const Center(child: Text('Something went wrong'));
        }
        if (!snapshot.hasData) {
          return const Center(child: CircularProgressIndicator());
        }
        final docs = snapshot.data!.docs;
        if (docs.isEmpty) {
          return const Center(child: Text('No students found'));
        }
        return ListView.builder(
          itemCount: docs.length,
          itemBuilder: (context, index) {
            final data = docs[index].data() as Map<String, dynamic>;
            return ListTile(
              title: Text(data['name'] ?? ''),
              subtitle: Text(
                'Roll: ${data['rollNumber'] ?? ''} | Course: ${data['course'] ?? ''}',
              ),
            );
          },
        );
      },
    ),
  );
}
```

10. Update an existing student record

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
```

```
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: UpdateStudentPage()));
}

class UpdateStudentPage extends StatefulWidget {
  const UpdateStudentPage({super.key});
  @override
  State<UpdateStudentPage> createState() => _UpdateStudentPageState();
}

class _UpdateStudentPageState extends State<UpdateStudentPage> {
  final documentId = TextEditingController();
  final newName = TextEditingController();

  Future<void> updateStudent() async {
    await FirebaseFirestore.instance
        .collection('students')
        .doc(documentId.text.trim())
        .update({'name': newName.text.trim()});
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Student updated')),
    );
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Update Student')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(children: [
        TextField(controller: documentId,
          decoration: const InputDecoration(
            labelText: 'Firestore Document ID')),
        TextField(controller: newName,
          decoration: const InputDecoration(labelText: 'New Name')),
        ElevatedButton(onPressed: updateStudent,
          child: const Text('Update')),
      ]),
    ),
  );
}
```

11. Delete a student record

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: DeleteStudentPage()));
}

class DeleteStudentPage extends StatefulWidget {
  const DeleteStudentPage({super.key});
  @override
  State<DeleteStudentPage> createState() => _DeleteStudentPageState();
}

class _DeleteStudentPageState extends State<DeleteStudentPage> {
  final documentId = TextEditingController();

  Future<void> deleteStudent() async {
    await FirebaseFirestore.instance
        .collection('students')
        .doc(documentId.text.trim())
        .delete();
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Student deleted')),
    );
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Delete Student')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(children: [
        TextField(controller: documentId,
          decoration: const InputDecoration(
```

```
            labelText: 'Firestore Document ID')),
        ElevatedButton(onPressed: deleteStudent,
          child: const Text('Delete')),
      ]),
    ),
  );
}
```

12. Filter students by course using where()

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: CourseStudentsPage()));
}

class CourseStudentsPage extends StatefulWidget {
  const CourseStudentsPage({super.key});
  @override
  State<CourseStudentsPage> createState() => _CourseStudentsPageState();
}

class _CourseStudentsPageState extends State<CourseStudentsPage> {
  final course = TextEditingController();
  String selectedCourse = '';

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Students by Course')),
    body: Column(children: [
      Padding(
        padding: const EdgeInsets.all(12),
        child: Row(children: [
          Expanded(child: TextField(controller: course,
            decoration: const InputDecoration(labelText: 'Enter course'))),
          IconButton(icon: const Icon(Icons.search),
            onPressed: () => setState(() =>
              selectedCourse = course.text.trim())),
        ]),
      ),
      Expanded(child: selectedCourse.isEmpty
        ? const Center(child: Text('Enter a course'))
        : StreamBuilder<QuerySnapshot>(
          stream: FirebaseFirestore.instance.collection('students')
            .where('course', isEqualTo: selectedCourse).snapshots(),
          builder: (context, snapshot) {
            if (!snapshot.hasData) {
              return const Center(child: CircularProgressIndicator());
            }
            return ListView(
              children: snapshot.data!.docs.map((doc) {
                final data = doc.data() as Map<String, dynamic>;
                return ListTile(
                  title: Text(data['name'] ?? ''),
                  subtitle: Text(data['course'] ?? ''),
                );
              }).toList(),
            );
          },
        )),
    ]),
  );
}
```

13. Display students in ascending order using orderBy()

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: SortedStudentsPage()));
}

class SortedStudentsPage extends StatelessWidget {
```

```
  const SortedStudentsPage({super.key});

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Students A to Z')),
    body: StreamBuilder<QuerySnapshot>(
      stream: FirebaseFirestore.instance.collection('students')
          .orderBy('name', descending: false).snapshots(),
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return const Center(child: Text('Error loading students'));
        }
        if (!snapshot.hasData) {
          return const Center(child: CircularProgressIndicator());
        }
        return ListView(
          children: snapshot.data!.docs.map((doc) {
            final data = doc.data() as Map<String, dynamic>;
            return ListTile(
              title: Text(data['name'] ?? ''),
              subtitle: Text('Roll: ${data['rollNumber'] ?? ''}'),
            );
          }).toList(),
        );
      },
    ),
  );
}
```

14. Notes application: add, view and delete notes for logged-in user

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: NotesPage()));
}

class NotesPage extends StatefulWidget {
  const NotesPage({super.key});
  @override
  State<NotesPage> createState() => _NotesPageState();
}

class _NotesPageState extends State<NotesPage> {
  final note = TextEditingController();
  final user = FirebaseAuth.instance.currentUser;

  CollectionReference get notes => FirebaseFirestore.instance
      .collection('users').doc(user!.uid).collection('notes');

  Future<void> addNote() async {
    if (note.text.trim().isEmpty) return;
    await notes.add({
      'text': note.text.trim(),
      'createdAt': FieldValue.serverTimestamp(),
    });
    note.clear();
  }

  @override
  Widget build(BuildContext context) {
    if (user == null) {
      return const Scaffold(
        body: Center(child: Text('Please log in first')),
      );
    }
    return Scaffold(
      appBar: AppBar(
        title: const Text('My Notes'),
        actions: [IconButton(
          icon: const Icon(Icons.logout),
          onPressed: () => FirebaseAuth.instance.signOut(),
        )],
      ),
      body: Column(children: [
        Padding(
          padding: const EdgeInsets.all(12),
          child: Row(children: [
            Expanded(child: TextField(controller: note,
              decoration: const InputDecoration(hintText: 'Write a note'))),
            IconButton(icon: const Icon(Icons.add), onPressed: addNote),
```

```
          ]),
        ),
        Expanded(child: StreamBuilder<QuerySnapshot>(
          stream: notes.orderBy('createdAt', descending: true).snapshots(),
          builder: (context, snapshot) {
            if (!snapshot.hasData) {
              return const Center(child: CircularProgressIndicator());
            }
            return ListView(
              children: snapshot.data!.docs.map((doc) {
                final data = doc.data() as Map<String, dynamic>;
                return ListTile(
                  title: Text(data['text'] ?? ''),
                  trailing: IconButton(
                    icon: const Icon(Icons.delete),
                    onPressed: () => notes.doc(doc.id).delete(),
                  ),
                );
              }).toList(),
            );
          },
        )),
      ]),
    );
  }
}
```

15. Student Management Application (Authentication + CRUD)

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MaterialApp(home: AuthGate()));
}

class AuthGate extends StatelessWidget {
  const AuthGate({super.key});
  @override
  Widget build(BuildContext context) => StreamBuilder<User?>(
    stream: FirebaseAuth.instance.authStateChanges(),
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Scaffold(
          body: Center(child: CircularProgressIndicator()));
      }
      return snapshot.hasData ? const StudentManager()
          : const LoginRegisterPage();
    },
  );
}

class LoginRegisterPage extends StatefulWidget {
  const LoginRegisterPage({super.key});
  @override
  State<LoginRegisterPage> createState() => _LoginRegisterPageState();
}

class _LoginRegisterPageState extends State<LoginRegisterPage> {
  final email = TextEditingController();
  final password = TextEditingController();

  Future<void> authenticate(bool isRegister) async {
    try {
      if (isRegister) {
        await FirebaseAuth.instance.createUserWithEmailAndPassword(
          email: email.text.trim(), password: password.text.trim());
      } else {
        await FirebaseAuth.instance.signInWithEmailAndPassword(
          email: email.text.trim(), password: password.text.trim());
      }
    } on FirebaseAuthException catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.message ?? 'Authentication failed')));
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Student App Login')),
    body: Padding(
```

```
      padding: const EdgeInsets.all(16),
      child: Column(children: [
        TextField(controller: email,
          decoration: const InputDecoration(labelText: 'Email')),
        TextField(controller: password, obscureText: true,
          decoration: const InputDecoration(labelText: 'Password')),
        ElevatedButton(onPressed: () => authenticate(false),
          child: const Text('Login')),
        TextButton(onPressed: () => authenticate(true),
          child: const Text('Register')),
      ]),
    ),
  );
}

class StudentManager extends StatefulWidget {
  const StudentManager({super.key});
  @override
  State<StudentManager> createState() => _StudentManagerState();
}

class _StudentManagerState extends State<StudentManager> {
  final name = TextEditingController();
  final roll = TextEditingController();
  final course = TextEditingController();

  CollectionReference get students =>
      FirebaseFirestore.instance.collection('students');

  Future<void> saveStudent({String? id}) async {
    final data = {
      'name': name.text.trim(),
      'rollNumber': roll.text.trim(),
      'course': course.text.trim(),
    };
    if (id == null) {
      await students.add(data);
    } else {
      await students.doc(id).update(data);
    }
    name.clear(); roll.clear(); course.clear();
  }

  void editStudent(String id, Map<String, dynamic> data) {
    name.text = data['name'] ?? '';
    roll.text = data['rollNumber'] ?? '';
    course.text = data['course'] ?? '';
    showDialog(context: context, builder: (dialogContext) => AlertDialog(
      title: const Text('Update Student'),
      content: const Text('Edit the fields in the form, then tap Update.'),
      actions: [
        TextButton(onPressed: () async {
          await saveStudent(id: id);
          if (dialogContext.mounted) Navigator.pop(dialogContext);
        }, child: const Text('Update')),
        TextButton(onPressed: () => Navigator.pop(dialogContext),
          child: const Text('Cancel')),
      ],
    ));
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(
      title: const Text('Student Management'),
      actions: [IconButton(
        icon: const Icon(Icons.logout),
        onPressed: () => FirebaseAuth.instance.signOut(),
      )],
    ),
    body: Column(children: [
      Padding(
        padding: const EdgeInsets.all(12),
        child: Column(children: [
          TextField(controller: name,
            decoration: const InputDecoration(labelText: 'Name')),
          TextField(controller: roll,
            decoration: const InputDecoration(labelText: 'Roll Number')),
          TextField(controller: course,
            decoration: const InputDecoration(labelText: 'Course')),
          ElevatedButton(onPressed: () => saveStudent(),
            child: const Text('Add Student')),
        ]),
      ),
      Expanded(child: StreamBuilder<QuerySnapshot>(
        stream: students.snapshots(),
        builder: (context, snapshot) {
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
```

```
          return ListView(
            children: snapshot.data!.docs.map((doc) {
              final data = doc.data() as Map<String, dynamic>;
              return ListTile(
                title: Text(data['name'] ?? ''),
                subtitle: Text(
                  '${data['rollNumber'] ?? ''} - ${data['course'] ?? ''}'),
                onTap: () => editStudent(doc.id, data),
                trailing: IconButton(
                  icon: const Icon(Icons.delete),
                  onPressed: () => students.doc(doc.id).delete(),
                ),
              );
            }).toList(),
          );
        },
      )),
    ]),
  );
}
```

Important notes

- Programs 2–7 require Email/Password authentication to be enabled in Firebase Console.
- Programs 8–15 use a Firestore collection named students. Program 14 stores notes under users/{userId}/notes.
- For programs 4, 7, 14 and 15, sign in first where applicable. The small demo login in Program 7 uses a sample account; replace it with y
- In Program 10 and 11, use the Firestore document ID of an existing student. You can find it in Firebase Console or use the list screen.
- Firestore security rules should restrict access in a real application. Do not leave test mode rules enabled for a deployed app.
- Each program is a separate exercise; do not paste multiple main() functions into the same main.dart file.